



BITS Pilani
DIGITAL
Excellence made yours

Professional AI/ML Programme

AWS LLM Foundations Lab

Build, Retrieve, Prompt & Deploy with Gemini Flash 2.5 on AWS

Hands-On Lab | BITS Pilani Digital

Session at a Glance

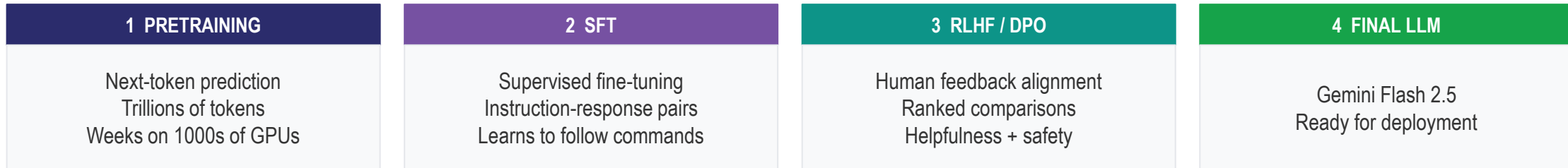
Six modules covering the full LLM application stack on AWS

6 Modules LLM foundations flow	1 Hands-On Lab Build -> retrieve -> deploy	8 RAG Steps End-to-end grounding	AWS Services S3 + Lambda + Bedrock	Gemini 2.5 Flash model examples
-----------------------------------	---	-------------------------------------	---------------------------------------	------------------------------------

Module 01	LLM Training & Artifacts - pretraining, SFT, RLHF/DPO, checkpoints -> S3
Module 02	Tool Calling - search, document retrieval, code interpreter at runtime
Module 03	Prompt Engineering - system prompts, few-shot, chain-of-thought, constraints
Module 04	RAG Pipeline - chunk -> embed -> FAISS/OpenSearch -> retrieve -> generate
Module 05	Generation Parameters - tokenisation, temperature, top-p, top-k, max tokens
Module 06	Classical vs LLMs - architecture, data, cost, latency, use-case mapping

Module 01 - LLM Training Pipeline

How Gemini Flash 2.5 was built - and what artifacts it produces



Training Artifacts Stored on S3

Artifact	Description	Size
model.safetensors	Learned weight matrices (all parameters)	1 GB - 700 GB
config.json	Architecture hyperparameters (layers, heads, dim)	~5 KB
tokenizer.json	Vocabulary + BPE merge rules	~3 MB
checkpoint-N/	Full snapshot at step N -> best checkpoint selection	Same as model
trainer_state.json	Loss curve, eval metrics at each step	~50 KB

Pretraining & Loss Tracking

Next-token prediction + checkpoint management on AWS S3

Pretraining Objective

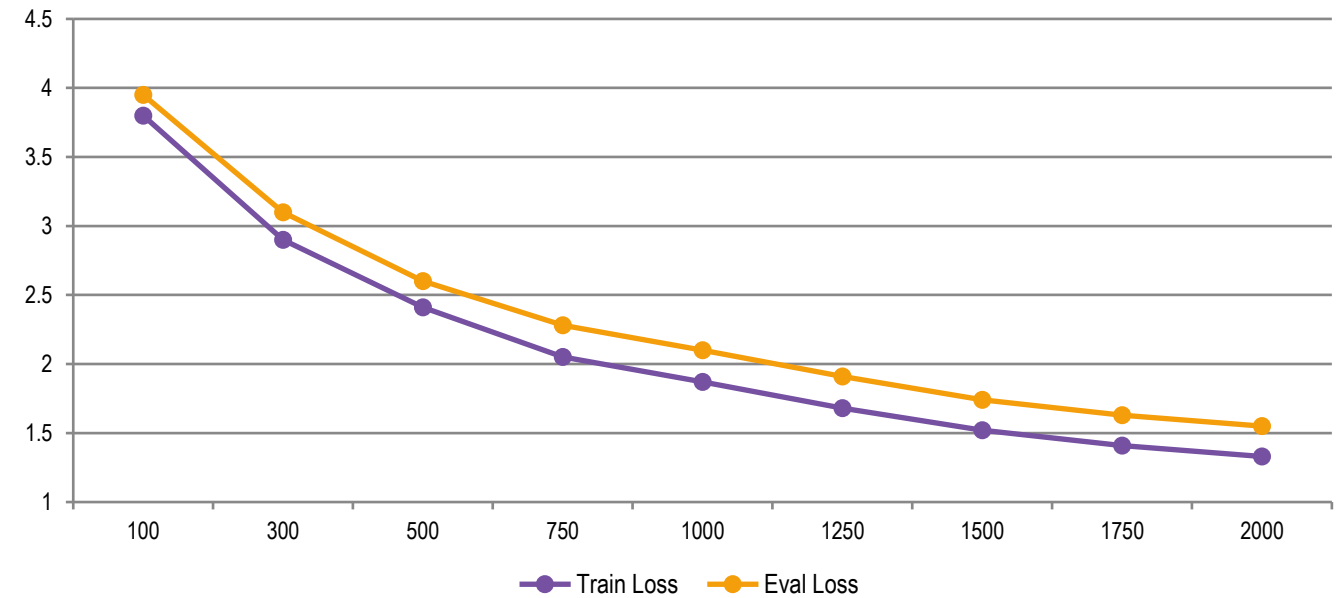
Given tokens: "The model was"
Predict: "trained"

Loss function: cross-entropy over vocabulary distribution at each position.

Lower perplexity means better language modelling capability.

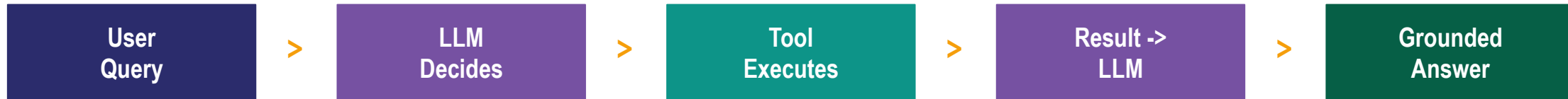
Scale: trillions of tokens x billions of parameters = emergent reasoning.

Training vs Eval Loss (SFT)



Module 02 - Tool Calling

LLMs that dynamically invoke search, documents and code at runtime



search_web(query)

```
result = search_web("Amazon Bedrock?")  
# Returns: KB content  
# AWS: Amazon Kendra  
# or OpenSearch
```

Searches a knowledge base or web for facts the LLM may not have in its weights.

get_document(name)

```
result = get_document("ml_policy.pdf")  
# Fetches from S3:  
# s3://bucket/docs/file  
# boto3.client("s3")
```

Retrieves specific policy docs, SLAs or reference files from S3 on demand.

run_code(code)

```
result = run_code("print(10000*0.08**5)")  
# Output: 46.888...  
# Captures stdout/stderr  
# Safe exec with sandbox
```

Executes Python code and returns stdout for calculations and data transforms.

Module 03 - Prompt Engineering

System prompts, few-shot learning, chain-of-thought and constrained output

Prompt Stack

SYSTEM PROMPT (developer-injected)

"You are a senior AWS solutions architect..."

FEW-SHOT EXAMPLES (optional)

"Ticket: S3 access denied" -> STORAGE

USER PROMPT (end-user input)

"How do I secure S3 with IAM roles?"

Zero-Shot

Ask directly, no examples. Works for clear, well-defined tasks with strong base models.

Few-Shot

Provide 2-5 input -> output examples. Controls format and style without retraining.

Chain-of-Thought

Use step-by-step reasoning instructions for multi-step math, logic and complex tasks.

Constraint

Define a JSON schema in the system prompt to force structured output for downstream parsing.

Module 04 - RAG Pipeline

Retrieval-Augmented Generation - grounding LLMs in your documents

INDEXING PHASE (offline)

1 Documents (PDF/TXT/HTML) from S3

2 Chunking -> 200 words, 40 overlap

3 Embedding -> 384-dim dense vectors

4 FAISS / OpenSearch k-NN index

RETRIEVAL + GENERATION (online)

5 User query -> embed with same model

6 Cosine similarity -> top-k chunks

7 Augmented prompt = context + query

8 Gemini Flash 2.5 -> grounded answer

AWS Production: S3 -> Lambda (chunk+embed) -> OpenSearch Serverless -> Bedrock -> API Gateway (streaming)

Module 05 - Generation Parameters

Tokenisation, temperature, top-p, top-k and max tokens - with examples



temperature - range: 0.0 - 2.0

Scales logits before softmax. T=0 -> greedy. T=2 -> very creative. Use 0.1-0.3 for factual, 0.8-1.2 for creative.

```
genai.GenerationConfig(temperature=0.2, max_output_tokens=500)
```

top_p (nucleus) - range: 0.0 - 1.0

Keep tokens until cumulative probability = P. P=0.9 -> nucleus of about 10-20 tokens. Smaller P = focused vocabulary.

```
genai.GenerationConfig(top_p=0.9, temperature=0.7)
```

top_k - range: 1 - infinity (typ. 40)

Hard vocabulary limit per step. K=1 = greedy. K=40 standard. Applied before top_p. Reduces rare-token noise.

```
genai.GenerationConfig(top_k=40, top_p=0.9, temperature=0.7)
```

max_output_tokens - range: 1 - 8192+

Hard stop on length. Billed per output token. Use about 20 for classify and about 2000 for code generation tasks.

```
genai.GenerationConfig(max_output_tokens=2000)
```


Module 06 - Classical ML vs LLMs

Architecture, data requirements, inference cost and when to choose each

Dimension	Classical ML	LLM (Gemini Flash 2.5)
Architecture	Linear, Tree, SVM, shallow MLP	Deep Transformer (self-attention)
Input	Fixed-size feature vector	Variable-length token sequence
Training data	Labelled rows (thousands+)	Trillions of tokens (unlabelled)
Training time	Minutes - days	Weeks - months on 1000s of GPUs
Parameters	10s - Millions	Billions - Trillions
Task setup	One model per task	One model, many tasks (prompting)
Explainability	High (SHAP, LIME, paths)	Low (attention maps only)
Inference latency	Microseconds - milliseconds	Milliseconds - seconds
AWS service	SageMaker built-in / SKLearn	Bedrock, SageMaker JumpStart

Use Classical ML: structured/tabular data, explainability required, ultra-low latency, cost-sensitive inference

Use LLMs: text/code/multimodal, multi-task, few labelled examples, complex reasoning and generation tasks

AWS Production Architecture

End-to-end LLM application stack on AWS services

Data Layer	S3 - Glue - Athena
Training Layer	SageMaker Training Jobs - Checkpoints -> S3
Embedding Layer	Bedrock Titan Embeddings - SageMaker Endpoint
Vector Store	OpenSearch Serverless (k-NN) - FAISS (local)
LLM Layer	Amazon Bedrock (Claude/Llama) - Gemini API
Serving Layer	Lambda - API Gateway - CloudFront (edge)
Observability	CloudWatch - X-Ray - Bedrock Model Eval

Security: IAM Roles (least privilege) - Secrets Manager (API keys) - VPC endpoints - KMS encryption at rest

Lab Complete

You now have the foundation to build, retrieve, prompt and deploy LLM applications on AWS.

Next Steps

Replace FAISS -> Amazon OpenSearch Serverless for production RAG

Deploy pipeline as Lambda + API Gateway with streaming responses

Add Bedrock Guardrails for PII redaction and content filtering

Evaluate RAG quality with RAGAS: faithfulness and context recall

Monitor costs with CloudWatch token usage dashboards

Fine-tune with SageMaker + Bedrock on domain-specific data